UNIVERSITATEA BABEŞ-BOLYAI

Facultatea de Ştiinţe Economice şi Gestiunea Afacerilor

Informatica Economica

Lucrare de licenţă

Coordonator ştiinţific,

Lector univ. dr. Florina Covaci

2025

UNIVERSITATEA BABEŞ-BOLYAI

Facultatea de Ştiinţe Economice şi Gestiunea Afacerilor

Denumirea completă a programului de studii

Lucrare de licenţă

Aplicație pentru generare automată de proiecte CRUD în ASP.NET Core

Coordonator ştiinţific,

Lector univ. dr. Florina Covaci

2025

Rezumat

În această lucrare de licență mi-am propus să prezint un sistem software care permite creearea automată a unui proiect în ASP.NET core, cu funcționalitate CRUD(Create, Read, Update, Delete). Plecând de la intenția de a reduce timpul pentru creearea unei aplicații automatizând un proces de programare repetitiv, am analizat modul în care este creată o aplicație de tip ASP.NET core și am reușit să identific procesul manual care se reproduce de fiecare dată la crearea unei noi aplicații. Pe baza acestei analize și utilizând cunoștințele dobândite în programarea cu .NET, am conceput și implementat un sistem capabil să automatizeze aceste procese repetitive, contribuind astfel la eficientizarea dezvoltării software.

Cuprins

Lista figurilor..... iv

Abrevieri ..... **v**
Introducere **1**
1. Metodologie **3**
1.2 Metodologia Waterfall 3
1.3 Tehnologii specifice 5
1.3.1 ASP.Net Core 5
1.3.2 Entity Framework 5
1.3.3 Angular 6
1.3.4 TypeScript 6
1.3.5 Model-View-Controller (MVC) 7
1.3.6 Interfață de programare a aplicațiilor (API) 9
1.3.7 JavaScript Object Notation (JSON) 10
1.3.8 Windows PowerShell 12
2. Analiza sistemului informatic **12**
2.1 Prezentarea problemei abordate 12
2.2 Motivatie 13
2.3 Elicitația cerințelor sistemului informatic 14
2.4 Cazurile de utilizare 17
3. Proiectarea și Implementarea sistemului informatic **20**
3.1 Structura generală a proiectului 20
3.2 Folderul Controllers 21
3.3 Folderul Service 25
3.3.1 Fisierul Functions.cs 27
3.3.2 Implementarea generatoarelor de cod 29
3.4 Generarea Fișierelor 30
3.4.1 ModelsGenerator.cs: 30
3.4.2 AppSettingGenerator 31
3.4.3 ControllerGenerator 31
3.4.4 DbContextGenerator 33
3.4.5 ProgramGenerator 34
4. Testarea **35**
5. Concluzii **40**
5.1 Rezultate obtinute 40
5.2 Directii viitoare 40
Bibliografie ..... Error! Bookmark not defined.
Glosar **42**

Lista figurilor

Figură 1 Metodologia Waterfall 4
Figură 2 MVC 8
Figură 3 MVC Repository 9
Figură 4 API 10
Figură 5 Arhitectura JSON 11
Figură 6 Diagrama cazurilor de utilizare 19
Figură 7 Metoda pentru generare AppDbContext.cs 21
Figură 8 Metoda pentru generarea Proiectului 22
Figură 9 Metoda pentru generarea AppSettingUpdate 22
Figură 10 Metoda pentru generarea Controllers.cs 23
Figură 11 Metoda pentru generarea Models.cs 23
Figură 12 Metoda pentru generarea Program.cs 23
Figură 13 Clasa GenerateAppDbContextModel 24
Figură 14 Scriptul pentru generarea proiectului 28
Figură 15 ModelsGenerator.cs 30
Figură 16 AppSettingsGenerator.cs 31
Figură 17 Solicitare HTTP de tip Get in controller 32
Figură 18 Solicitare HTTP de tip Post in controller 32
Figură 19 Solicitare HTTP de tip Put in controller 32
Figură 20 Solicitare HTTP de tip Delete in controller 33
Figură 21 DbContextGenerator 33
Figură 22 ProgramGenerator 35
Figură 23 Test Case 1 38
Figură 24 Rezultat Test Case 1 39

Abrevieri

CRUD – Create, Read, Update, Delete

MVC – Model-View-Controller

API – Application Programming Interface

EF – Entity Framework

ORM – Object-Relational Mapper

SQL – Structured Query Language

SPA – Single Page Application

HTML – HyperText Markup Language

JSON – JavaScript Object Notation

HTTP – HyperText Transfer Protocol

AI – Artificial Intelligence

IDE – Integrated Development Environment

REST – Representational State Transfer

DTO – Data Transfer Object

XML – eXtensible Markup Language

STLC – Software Testing Life Cycle

QA – Quality Assurance

Introducere

În zilele noastre, viața pe care o traim e bazata pe eficiență și productivitate . Așadar, lucrurile sunt într-o continuă schimbare pentru că trăim o nouă realitate. În contextul work-from-home, apar mai multe elemente de distragere a atenției, astfel, abilitatea de a-ți gestiona timpul în mod eficient face tranziția de la eșec la succes. Când vine vorba de job, fie că suntem angajați la o firma, freelanceri sau suntem antreprenori, cuvintele de ordine sunt „productivitate” și „eficiență”. În acest mod, nevoia de a economisi timp și de a lucra mai inteligent, nu neapărat mai mult devine tot mai mare, soluțiile tehnologice, în special cele care vizează automatizarea sarcinilor repetitive, apar ca un răspuns firesc la această nevoie. Așadar, eficiența și productivitatea nu mai depind exclusiv de efortul uman brut, ci și de capacitatea de a utiliza instrumente care optimizează lucrul de zi cu zi.

Adesea, oamenilor li se creează părerea că automatizarea are in plan sa fure locurile de muncă, dar daca privim mai atent putem observa faptul că automatizarea nu fură ci sporește productivitatea si eficiența firmelor si a companiilor prin faptul că înlocuiește anumite procese manuale repetitive și obositoare cu roboți sau mașinarii care chiar sporesc venitul și îi ajută pe oameni să se dezvolte o dată cu tehnologia. În ultimii ani volumul de munca tot crește, de aceea nevoia ca oamenii să se adapteze la dezvoltarea tehnologiilor este esențială. Deci ca obiectiv principal automatizarea asre scopul de a sporii productivitatea și de a oferii suport oamenilor pentru a înlocui sarcinile repetitive și obositoare, lucrul acesta ajutând la reducerea timpului de execuție și asigură performanțe maxime.

În cadrul acestui proces de dezvoltare software utilizând .NET, orice programator este familiarizat cu pașii necesari realizarii unui nou proiect și cu pregătirile manuale care trebuie realizate înainte de a putea începe implementarea propriu-zisă a funcționalităților aplicației pe care vrea sa o creeze. Etapele acestea, deși aparent simple si care poate pentru un profesionist nu mai sunt un discomfor, consumă timp prețios și se repetă identic pentru fiecare nouă aplicație creată, ceea ce duce la scăderea eficienței si la creșterea costurilor de dezvoltare ale aplicației dorite. În contextul acesta, apare tot mai pregnantă necesitatea unui sistem care să automatizeze procesul de creare a unei structuri de bază pentru proiectele ASP.NET Core, o asemenea soluție nu doar că optimizează timpul alocat fazei inițiale de dezvoltare, dar contribuie și la reducerea erorilor umane și, implicit, la scăderea costurilor asociate proiectelor software.

Astfel, aplicația pe care am dezvoltat-o oferă utilizatorului posibilitatea de a genera rapid și eficient un proiect nou de tip ASP.NET Core, echipat cu funcționalități complete de tip CRUD (Create, Read, Update, Delete), precum și cu structura de tabele necesare gestionării datelor. Prin intermediul unei interfețe intuitive, utilizatorul poate specifica parametrii de bază ai proiectului, iar aplicația se ocupă automat de generarea codului sursă, a fișierelor de configurare și a structurilor

de date necesare funcționării aplicației. Acest proces reduce considerabil timpul de dezvoltare, elimină erorile frecvente apărute în faza de inițializare și oferă o bază solidă pentru extinderea ulterioară a aplicației în funcție de cerințele specifice ale fiecărui proiect. Soluția propusă este astfel un instrument util pentru dezvoltatorii .NET, contribuind la eficientizarea muncii lor prin eliminarea etapelor repetitive și crearea unui mediu de lucru standardizat și scalabil.

Lucrarea de față își propune să exploreze în profunzime procesul de dezvoltare al aplicației propuse, evidențiind etapele parcurse de la idee la implementare, precum și deciziile tehnice care au stat la baza arhitecturii și funcționalităților sale. Totodată, lucrarea oferă o perspectivă detaliată asupra modului în care utilizatorii interacționează cu aplicația, analizând experiența acestora din punctul de vedere al ușurinței în utilizare, eficienței și utilității practice. Prin această abordare, se urmărește nu doar descrierea unei soluții tehnice, ci și evidențierea modului în care aceasta răspunde unor nevoi reale din domeniul dezvoltării software, contribuind la îmbunătățirea proceselor de lucru și la optimizarea resurselor implicate în crearea de aplicații web moderne.

1. Metodologie

1.2 Metodologia Waterfall

Metodologia Waterfall, cunoscută și sub denumirea de model în cascadă, reprezintă una dintre cele mai tradiționale și consacrate abordări în realizarea si gestionarea proiectelor, fiind utilizată cu precădere în contexte în care cerințele sunt bine definite încă de la început. Această metodologie urmează un model liniar și secvențial de dezvoltare, în care fiecare etapă trebuie finalizată complet înainte de a trece la următoarea. Structura clasică a modelului Waterfall este compusă din următoarele faze:

- analiza cerințelor
- proiectarea sistemului
- implementarea
- testarea
- lansarea (deployment)
- întreținerea

Această succesiune clară de etape permite o planificare riguroasă, o distribuire eficientă a resurselor și un control precis al progresului proiectului.

Unul dintre principiile fundamentale ale acestei metodologii este presupunerea că toate cerințele sunt cunoscute de la începutul proiectului și că acestea nu se vor modifica semnificativ pe parcurs. Din acest motiv, metodologia Waterfall este recomandată în proiectele unde schimbările în timpul dezvoltării sunt dificil de gestionat sau costisitoare, cum ar fi în inginerie, construcții sau producție industrială.Prin structura sa predictibilă și documentația detaliată aferentă fiecărei etape, Waterfall oferă o soluție stabilă și controlabilă, mai ales în proiectele cu complexitate tehnică ridicată și cu un grad redus de incertitudine.

Astfel, am ales metodologia Waterfall deoarece am considerat organizarea și cunoașterea detaliată a fiecarui pas un element essențial în dezvoltarea aplicației mele, pentru o mai bună organizare și productivitate. Am identificat câteva benefici pe care le are un programator dacă folosește această metodă:

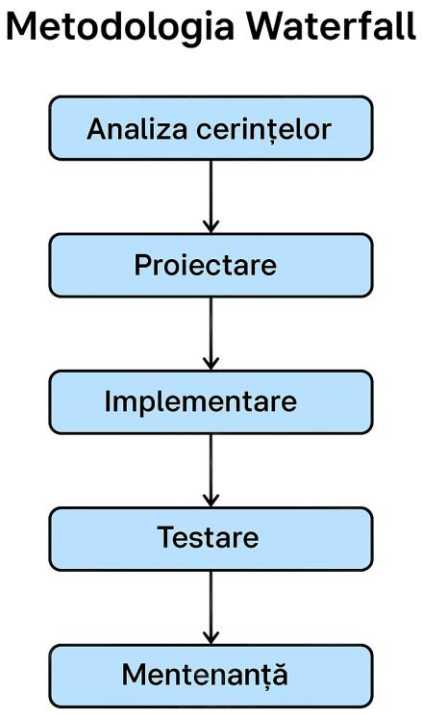
1. **Structură și ordine:** Waterfall oferă etape și programe clar definite, permițând planificarea și monitorizarea eficientă a progresului.

2. **Transparență:** Fiecare etapă a proiectului este documentată în detaliu, permițând urmărirea ușoară a progresului și identificarea problemelor.

3. **Minimizarea riscurilor:** Definirea precisă a cerințelor la începutul proiectului minimizează riscul de modificări în timpul execuției.

4. **Predictibilitate:** Planurile și programele detaliate permit determinarea exactă a costurilor și a termenelor proiectului.
5. **Controlul:** Waterfall facilitează gestionarea resurselor și controlul bugetului, care sunt esențiale pentru succesul proiectului.

Procesul de dezvoltare a aplicației a început cu o analiză detaliată a cerințelor, în cadrul căreia am colectat toate informațiile relevante și am definit în mod clar obiectivele proiectului. Ulterior, am elaborat un plan de acțiune bine structurat, care a inclus un calendar al etapelor de lucru, precum și alocarea resurselor necesare fiecărei faze. După această etapă pregătitoare, am trecut la faza de proiectare, în care am urmărit cu atenție toate componentele sistemului, asigurându-mă că soluțiile tehnice propuse corespund cerințelor funcționale și nefuncționale identificate anterior. Finalizarea acestei etape a permis trecerea la dezvoltarea propriu-zisă a aplicației, desfășurată conform planului stabilit. Testarea a reprezentat o etapă esențială, prin care am verificat funcționalitatea, acuratețea și stabilitatea componentelor dezvoltate, pentru a mă asigura că toate cerințele specificate în documentație sunt pe deplin respectate. În final, am ajuns în etapa de implementare completă, în care produsul a fost finalizat și pregătit pentru utilizare. (Dominik)



Figură 1 Metodologia Waterfall

1.3 Tehnologii specifice

În această secțiune voi detalia tehnologiile utilizate în procesul de dezvoltare a aplicației web, evidențiind rolul fiecărei componente în implementarea funcționalităților. Vor fi analizate limbajele de programare alese, framework-urile asociate, bibliotecile utilizate, precum și mediul de dezvoltare, toate acestea contribuind esențial la realizarea aplicației într-un mod eficient și scalabil.

1.3.1 ASP.Net Core

Framework-ul utilizat pentru dezvoltarea aplicației este ASP.NET Core, o platformă modulară, open-source, destinată construirii aplicațiilor web moderne. Reprezentând o reproiectare completă a vechiului framework ASP.NET, ASP.NET Core unifică într-un model de programare coerent funcționalitățile oferite anterior de ASP.NET MVC și ASP.NET Web API, facilitând astfel dezvoltarea mai eficientă a aplicațiilor web. Deși este construit pe o arhitectură nouă, ASP.NET Core păstrează o compatibilitate conceptuală ridicată cu versiunile anterioare ale ASP.NET, ceea ce îl face ușor de adoptat de către dezvoltatorii familiarizați cu ecosistemul Microsoft. Un avantaj semnificativ al acestui framework îl reprezintă suportul pentru versionare side-by-side, ceea ce permite rularea simultană, pe aceeași mașină, a mai multor aplicații dezvoltate cu versiuni diferite de ASP.NET Core — un aspect care nu era posibil în versiunile tradiționale ale ASP.NET. Inițial, ASP.NET Core a oferit compatibilitate atât cu .NET Framework (disponibil exclusiv pe Windows), cât și cu .NET Core (cross-platform). Începând cu versiunea ASP.NET Core 3.0, suportul pentru .NET Framework a fost retras, consolidându-se astfel orientarea către o arhitectură multiplatformă, capabilă să ruleze pe Windows, Linux și macOS. (Microsoft, ASP.NET Core, 2024)

1.3.2 Entity Framework

Entity Framework (EF) este un framework de tip Object-Relational Mapper (ORM) dezvoltat de Microsoft, conceput pentru a facilita lucrul cu baze de date relaționale în aplicațiile dezvoltate pe platforma .NET. EF permite programatorilor să interacționeze cu datele dintr-o bază relațională folosind obiecte din codul aplicației, eliminând necesitatea scrierii manuale a interogărilor SQL și a logicii de mapare între tabele și obiecte. Această abordare aduce un avantaj major în termeni de productivitate și claritate a codului, întrucât dezvoltatorii pot lucra direct cu entități și colecții de obiecte, manipulând datele într-un mod natural, specific programării orientate pe obiect. EF gestionează automat conversia între structurile relaționale din baza de date și structurile obiectuale din codul aplicației. Unul dintre aspectele fundamentale pe care le abordează Entity Framework este „nepotrivirea de impedanță obiect-relațională” (object-relational impedance mismatch) — o

problemă comună în dezvoltarea software, care apare atunci când modelul obiectual din aplicație diferă semnificativ de modelul relațional al bazei de date. EF oferă un mecanism de mapare eficient între aceste două lumi, simplificând sincronizarea datelor și reducând considerabil complexitatea logicii de acces. (Microsoft, Entity Framework, 2022)

În cadrul proiectului realizat, EF a fost utilizat pentru a crea automat clasele entitate corespunzătoare tabelelor definite de utilizator și pentru a permite manipularea datelor prin operații CRUD. Acest lucru a dus la o integrare simplă și eficientă a bazei de date cu backend-ul ASP.NET Core, contribuind la un cod mai curat, mai scalabil și mai ușor de întreținut. (Microsoft, Entity Framework (EF))

1.3.3 Angular

Angular este un framework structural JavaScript open-source, dezvoltat de Google, utilizat pentru construirea aplicațiilor web de tip Single Page Application (SPA). Aceste aplicații se încarcă o singură dată în browser, iar actualizarea conținutului are loc dinamic, fără reîncărcarea completă a paginii. Angular este construit pe baza unui sistem modular de componente, fiecare reprezentând o parte a interfeței aplicației și fiind compusă dintr-un fișier HTML, un fișier TypeScript și stiluri asociate. Framework-ul permite extinderea sintaxei HTML prin directive și binding-uri, facilitând crearea de interfețe declarative și dinamice. Printre facilitățile cheie oferite de Angular se numără data binding-ul (sincronizarea automată între date și interfață) și dependency injection-ul (gestionarea eficientă a serviciilor). Aceste mecanisme reduc semnificativ volumul de cod necesar și îmbunătățesc organizarea aplicației.

În proiectul realizat, Angular a fost folosit pentru crearea interfeței de utilizator, permițând introducerea datelor necesare generării backend-ului și comunicarea cu serverul ASP.NET Core prin intermediul API-urilor. (Development)

1.3.4 TypeScript

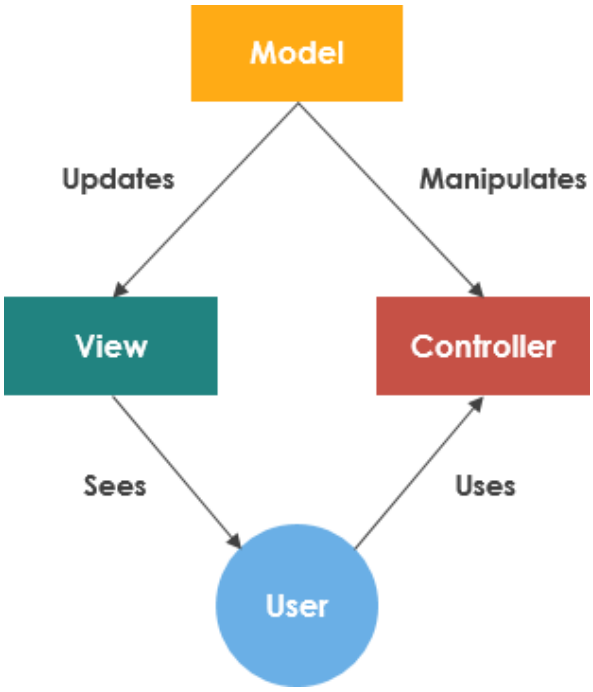
TypeScript este un limbaj de programare dezvoltat de Microsoft, conceput pentru a extinde funcționalitatea JavaScript prin introducerea unui sistem de tipare statice. Acesta este destinat dezvoltării de aplicații de mari dimensiuni, oferind suport avansat pentru verificarea tipurilor la compilare, ceea ce contribuie la detectarea timpurie a erorilor și la creșterea robusteții codului. Spre deosebire de JavaScript, care este un limbaj dinamic, TypeScript permite definirea explicită a tipurilor pentru variabile, funcții, clase și alte structuri, facilitând astfel o dezvoltare mai controlată și predictibilă. Cu toate acestea, TypeScript este pe deplin compatibil cu standardele JavaScript, fiind transcompilat într-un cod JavaScript clar și lizibil, care poate fi executat în orice browser modern, pe orice sistem de operare sau platformă. Această compatibilitate și flexibilitate îl fac ideal pentru proiectele complexe de tip frontend, cum ar fi cele dezvoltate cu Angular, unde

structura clară și scalabilitatea codului sunt esențiale. TypeScript nu doar îmbunătățește experiența de dezvoltare, ci și facilitează mentenanța și extinderea ulterioară a aplicațiilor. (npmjs.com, 2025)

1.3.5 Model-View-Controller (MVC)

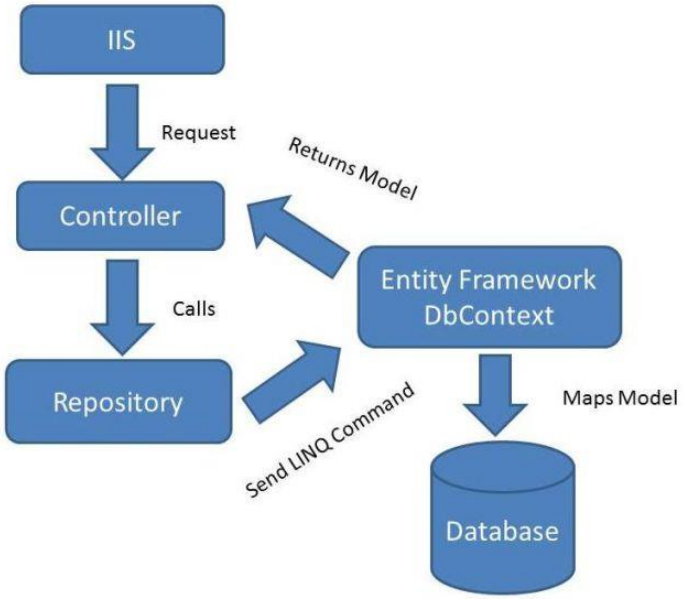
MVC (Model-View-Controller) este un model în designul software utilizat în mod obișnuit pentru a implementa interfețe utilizator, date și logica de control. Acesta pune accent pe separarea dintre logica de business a software-ului și afișare. Această „separare a preocupărilor" asigură o mai bună diviziune a muncii și o întreținere îmbunătățită. Alte modele de design se bazează pe MVC, cum ar fi MVVM (Model-View-Viewmodel), MVP (Model-View-Presenter) și MVW (Model-View-Whatever).

Modelul reprezintă componenta centrală a arhitecturii MVC care definește structura și logica datelor pe care le gestionează aplicația. Acesta stabilește ce tipuri de informații sunt necesare, cum sunt organizate și cum pot fi accesate sau modificate. În esență, modelul reflectă starea aplicației și servește drept intermediar între sursa de date (de exemplu, o bază de date) și celelalte componente ale arhitecturii. Atunci când datele din model se modifică fie ca urmare a unei acțiuni a utilizatorului, fie printr-o actualizare externă modelul are responsabilitatea de a notifica celelalte componente relevante. În mod obișnuit, notifică vizualizarea (View), pentru ca aceasta să poată actualiza automat afișajul și să reflecte noile date. Acest mecanism asigură sincronizarea dintre interfața cu utilizatorul și starea internă a aplicației, fără a fi necesară intervenția manuală. În unele cazuri, modificările din model pot necesita și o actualizare a controlerului (Controller), în special atunci când este nevoie de o logică diferită pentru gestionarea noii stări a aplicației. De exemplu, dacă o anumită acțiune duce la o eroare în model (cum ar fi o validare eșuată), controlerul poate fi implicat pentru a decide cum trebuie gestionată situația și ce feedback să fie transmis utilizatorului. (DevelopmentFactory, 2024)



Figură 2 MVC

View este responsabila de afișarea datelor provenite din Model către utilizator și de colectarea interacțiunilor utilizatorului. Aceasta are un rol pasiv, neinteracționând direct cu Modelul. În schimb, primește informațiile de afișat prin intermediul Controlerului sau al unui mecanism de legare a datelor și transmite acțiunile utilizatorului (cum ar fi clicuri, completarea unor formulare etc.) către Controler pentru a fi procesate. Astfel, View-ul acționează ca o interfață între utilizator și restul aplicației, fără a conține logică de procesare sau de manipulare a datelor. (GeeksForgeeks, 2025)



Figură 3 MVC Repository

Controlerul (Controller) acționează ca un intermediar între Model și Vizualizare, fiind responsabil de coordonarea fluxului de date și acțiuni în cadrul aplicației. El preia datele introduse de utilizator prin intermediul View-ului, le validează, aplică logica necesară și actualizează Modelul în funcție de aceste intrări. De asemenea, după ce Modelul este modificat, Controlerul poate determina ce Vizualizare trebuie afișată și cum să reflecte schimbările. În esență, Controller-ul conține logica aplicației, gestionând transformările datelor, validările și deciziile legate de modul în care interacționează utilizatorul cu aplicația. (GeeksForgeeks.org, 2025)

1.3.6 Interfață de programare a aplicațiilor (API)

Interfața de programare a aplicațiilor (API – Application Programming Interface) reprezintă un set de definiții, reguli și protocoale care permit comunicarea dintre două aplicații software, fără a necesita intervenția directă a utilizatorului. API-urile oferă o modalitate standardizată prin care diverse sisteme pot colabora, facilitând schimbul de date și comenzi într-un mod controlat, sigur și eficient.

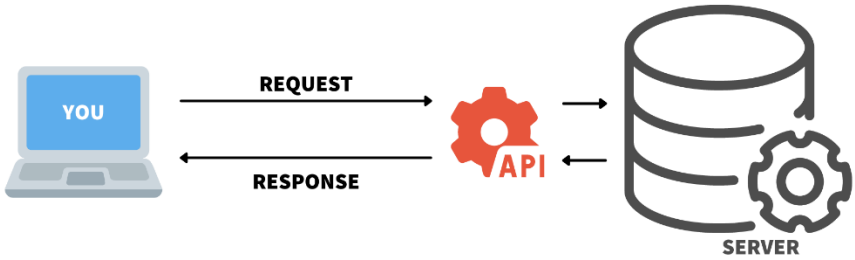
Din punct de vedere tehnic, un API este o colecție de funcții, metode sau proceduri prin intermediul cărora o aplicație poate **accesa funcționalitățile sau resursele** unei alte aplicații, platforme sau servicii. Prin abstractizarea detaliilor de implementare, API-ul acționează ca o punte de legătură între două componente software, permițându-le să interacționeze fără ca una să cunoască structura internă a celeilalte.

Această separare a responsabilităților oferă multiple avantaje:

- **Modularitate și reutilizare** a codului;
- **Scalabilitate**, deoarece componentele pot fi extinse sau înlocuite fără a afecta întregul sistem;
- **Interoperabilitate între tehnologii diferite**, lucru esențial în arhitecturile moderne distribuite.

API-urile sunt omniprezente în dezvoltarea software modernă, de la aplicații web și mobile până la servicii cloud sau sisteme enterprise. Ele pot funcționa local, într-un sistem închis, sau pot fi expuse ca **API-uri web (Web APIs)**, accesibile prin protocoale de rețea precum HTTP, fiind deosebit de utile în contextul aplicațiilor front-end care comunică cu un backend.

În cadrul aplicației dezvoltate în această lucrare, API-ul joacă un rol central, facilitând comunicarea între interfața de utilizator realizată în Angular și backend-ul ASP.NET Core. Operațiunile de tip CRUD sunt realizate prin intermediul apelurilor la API, utilizând formate de date precum JSON pentru transmiterea informațiilor între client și server. Astfel, API-ul devine componenta fundamentală care asigură funcționarea logicii de business și coerența fluxului de date în cadrul aplicației. (Ilie, 2024)



Figură 4 API

1.3.7 JavaScript Object Notation (JSON)

JSON (JavaScript Object Notation) este un format standard, bazat pe text, utilizat pentru reprezentarea datelor structurate, având la bază sintaxa obiectelor din JavaScript. Fiind ușor de

citit de către oameni și simplu de procesat de către mașini, JSON a devenit unul dintre cele mai populare formate pentru schimbul de date între client și server în aplicațiile web moderne.

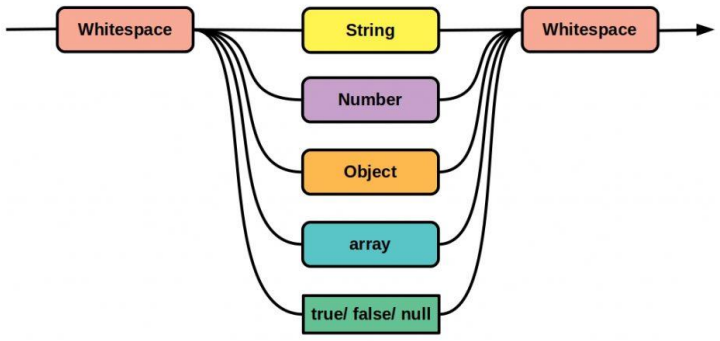
JSON este utilizat frecvent pentru **transmiterea datelor între aplicații**, în special în arhitecturile de tip client–server. Un exemplu obișnuit este trimiterea datelor de la server către interfața de utilizator (frontend), unde acestea sunt interpretate și afișate, sau invers – transmiterea datelor introduse de utilizator către backend, pentru procesare și stocare.

Structura JSON este simplă și se bazează pe două concepte fundamentale:

- **Obiecte**: o colecție de perechi cheie–valoare, echivalente cu dicționarele sau structurile din alte limbaje.
- **Liste (array-uri)**: o colecție ordonată de valori, similare cu matricile sau listele din majoritatea limbajelor de programare.

Aceste structuri de date sunt recunoscute și suportate nativ de majoritatea limbajelor moderne, ceea ce face ca JSON să fie un **format ideal pentru interoperabilitate** între sisteme dezvoltate în tehnologii diferite.

În contextul aplicației dezvoltate în această lucrare, JSON a fost utilizat pentru **transmiterea datelor între interfața realizată în Angular și backend-ul ASP.NET Core**, în special pentru operațiunile CRUD. De asemenea, datorită suportului nativ în JavaScript și C#, conversia datelor din și în format JSON este eficientă și fiabilă, contribuind la un flux de lucru coerent și ușor de întreținut. (Mozzila, n.d.)



Figură 5 Arhitectura JSON

1.3.8 Windows PowerShell

PowerShell este o interfață de comandă modernă care include cele mai bune caracteristici ale altor interfețe de comandă populare. Spre deosebire de majoritatea interfețelor de comandă care acceptă și returnează doar text, PowerShell acceptă și returnează obiecte .NET. Interfața de comandă include următoarele caracteristici:

- Istoric robust al liniei de comandă
- Completare cu tabulatorul și predicție a comenzilor (vezi about_PSReadLine)
- Acceptă aliasuri de comenzi și parametri
- Canal pentru înlănțuirea comenzilor
- Sistem de ajutor în consolă , similar manpaginilor Unix

Ca limbaj de scripting, PowerShell este utilizat în mod obișnuit pentru automatizarea gestionării sistemelor. De asemenea, este utilizat pentru a construi, testa și implementa soluții, adesea în medii CI/CD. PowerShell este construit pe .NET Common Language Runtime (CLR). Toate intrările și ieșirile sunt obiecte .NET. Nu este nevoie să analizați textul de ieșire pentru a extrage informații din ieșire. Limbajul de scripting PowerShell include următoarele caracteristici:

- Extensibil prin funcții , clase , scripturi și module
- Sistem de formatare extensibil pentru o ieșire ușoară
- Sistem de tipuri extensibil pentru crearea de tipuri dinamice
- Suport încorporat pentru formate de date comune precum CSV , JSON și XML (Microsoft, What is PowerShell?, 2024)

2. Analiza sistemului informatic

În acest capitol îmi propun să expun problematica ce a stat la baza alegerii dezvoltării unui sistem pentru generarea automată a backend-ului. De asemenea, voi detalia contextul în care poate fi aplicată soluția propusă, evidențiind utilitatea acesteia în practică. Nu în ultimul rând, voi prezenta motivația personală și profesională care a stat la baza alegerii acestei teme pentru lucrarea de licență.

2.1 Prezentarea problemei abordate

Această lucrare propune un sistem informatic dedicat celor care doresc să evite reconstrucția manuală a aceloraşi componente backend. Soluția propusă constă într-o aplicație care automatizează procesul de generare a unui sistem CRUD (Create, Read, Update, Delete) cu tabele

în ASP.NET Core, eliminând necesitatea realizării acestor pași de la zero pentru fiecare proiect nou. Prin utilizarea acestui sistem, programatorii pot economisi timp prețios, pot diminua volumul de muncă și pot reduce costurile asociate dezvoltării aplicațiilor web. În esență, aplicația își propune să devină un instrument de sprijin real pentru dezvoltatori, permițându-le să se concentreze pe aspectele unice și complexe ale proiectelor, care nu pot fi automatizate, contribuind astfel la o experiență de programare mai plăcută și mai eficientă.

Una dintre dificultățile frecvent întâlnite de programatori, în special în etapa de inițializare a proiectelor web, este lipsa unor aplicații specializate care să automatizeze procesul de generare a backend-ului, în special pentru framework-ul ASP.NET Core. Deși pentru alte tehnologii, precum Laravel (PHP) sau Django (Python), există instrumente și generatoare care creează rapid structura de bază a aplicației, pentru ASP.NET Core oferta este semnificativ mai limitată.

O simplă căutare pe internet relevă faptul că există foarte puține soluții care să ofere o automatizare completă a procesului de creare a unui sistem CRUD funcțional, cu tabele și operațiuni esențiale (Create, Read, Update, Delete), integrate într-o arhitectură coerentă. O altă problemă importantă constă în faptul că majoritatea resurselor și soluțiilor existente pentru generarea automată a unui backend sunt fie incomplete, fie presupun un grad ridicat de intervenție manuală din partea dezvoltatorului. De cele mai multe ori, aceste instrumente oferă doar generarea parțială a structurii aplicației — cum ar fi clasele model sau scheletul controlerelor — fără a asigura o integrare completă între componente, validări, conexiuni cu baza de date și funcționalități CRUD funcționale cap-coadă. Această situație obligă dezvoltatorii să intervină suplimentar în cod, să configureze manual diverse fișiere, să adauge logică de legătură și să gestioneze dependințe, aspecte care consumă timp și pot duce la erori. În consecință, ideea de eficientizare a muncii, care ar trebui să stea la baza utilizării unor astfel de instrumente, este în mare parte compromisă. Prin urmare, apare o nevoie clară pentru o soluție care să ofere un grad cât mai înalt de automatizare, dar care să fie totodată flexibilă, completă și ușor de utilizat, reducând astfel semnificativ efortul necesar în etapele inițiale ale dezvoltării aplicațiilor web.

Această lipsă de unelte accesibile și flexibile pentru ASP.NET Core creează un gol semnificativ în ecosistemul .NET, ceea ce justifică necesitatea dezvoltării unei aplicații care să răspundă concret acestei nevoi — automatizarea procesului de generare a backend-ului. Astfel, soluția propusă în această lucrare vine să acopere un vid real și să ofere un instrument valoros dezvoltatorilor .NET.

2.2 Motivatie

Scopul principal al dezvoltării acestei aplicații a fost acela de a le oferi programatorilor o soluție practică pentru eficientizarea timpului și a efortului depus în procesul de creare a aplicațiilor ASP.NET Core. Motivația a pornit din dorința de a elimina pașii manuali și repetitivi specifici configurării inițiale a unui proiect, prin automatizarea completă a acestora. În plus, aplicația a fost

concepută astfel încât să nu necesite intervenții asupra codului sursă generat, reducând la minimum inputul manual și asigurând o experiență cât mai confortabilă și intuitivă pentru utilizator.

Observând numărul extrem de redus de aplicații dedicate generării automate a backend-ului pentru proiecte de tip ASP.NET Core, am considerat necesară dezvoltarea unei soluții care să vină în sprijinul comunității de programatori, dar și al meu personal, în vederea eficientizării procesului de lucru în proiectele viitoare. Această aplicație răspunde unei nevoi reale din practică: economisirea timpului și eliminarea etapelor repetitive, adesea predispuse la erori. Desigur, nu poate fi ignorată apariția inteligenței artificiale ca alternativă modernă de generare a codului. Totuși, în urma testării mai multor modele AI, am constatat că, deși acestea pot genera coduri complexe și bine structurate, nu reușesc, în cele mai multe cazuri, să ducă la finalizarea completă a unui proiect funcțional fără intervenții manuale semnificative. În plus, pentru un dezvoltator mai puțin experimentat, interpretarea și ajustarea codului generat de AI poate deveni mai degrabă o sursă de confuzie decât un ajutor real. Prin contrast, aplicația propusă în această lucrare își propune să automatizeze integral procesul de creare a backend-ului, fără a necesita editări suplimentare sau cunoștințe avansate de programare, oferind astfel o alternativă clară, controlată și fiabilă la soluțiile actuale.

Un alt motiv important care a stat la baza alegerii acestei teme este pasiunea personală pentru programare, în special în cadrul ecosistemului .NET. Această aplicație nu este doar un exercițiu tehnic, ci și o expresie a interesului meu constant pentru dezvoltarea de soluții software care aduc valoare reală și pot fi utilizate practic. Prin realizarea acestui proiect, am avut ocazia să-mi consolidez cunoștințele în programarea backend și să îmi dezvolt capacitatea de a integra diferite concepte și tehnologii într-un sistem coerent și funcțional. Totodată, acest demers reprezintă un aport semnificativ în dezvoltarea mea profesională, având un impact pozitiv asupra portofoliului și CV-ului meu, demonstrând nu doar abilități tehnice, ci și inițiativă, autonomie și capacitate de rezolvare a problemelor reale..

2.3 Elicitația cerințelor sistemului informatic

O etapă fundamentală în procesul de analiză a unui sistem informatic constă în identificarea, colectarea și clarificarea nevoilor reale ale utilizatorilor pe care aplicația trebuie să le satisfacă. Această etapă este cunoscută ca elicitația cerințelor și reprezintă baza întregului proces de proiectare și dezvoltare.

În cazul aplicației dezvoltate în cadrul acestei lucrări, destinată generării automate a unui backend ASP.NET Core cu funcționalități CRUD, grupul-țintă este relativ restrâns, fiind compus în principal din programatori, studenți în domeniul informaticii sau persoane pasionate de programare care caută metode mai rapide și mai eficiente de a-și inițializa proiectele.

Scopul acestei etape a fost acela de a înțelege în profunzime:

- care sunt așteptările utilizatorilor de la aplicație,
- ce funcționalități sunt esențiale pentru a răspunde nevoilor reale,
- ce limitări tehnice sau funcționale pot apărea în utilizarea practică a soluției,
- în ce contexte specifice va fi utilizată aplicația (ex. în dezvoltare rapidă, la proiecte academice sau profesionale).

Pentru a defini cât mai clar cerințele, procesul de elicitație s-a concentrat pe o problemă frecvent întâlnită în practică: nevoia de a automatiza pașii manuali și redundanți care apar în etapa inițială a dezvoltării aplicațiilor web, mai ales în cadrul proiectelor .NET.

În vederea obținerii unor cerințe cât mai precise, am utilizat două metode de elicitare:

Interviul formal:

Au fost adresate întrebări structurate unor persoane cu experiență în dezvoltarea de aplicații ASP.NET Core, pentru a identifica pașii pe care îi repetă constant la începutul fiecărui proiect și ce îmbunătățiri ar considera utile.

- Care sunt primii pași pe care îi urmezi atunci când începi un nou proiect ASP.NET Core?
- Există o structură standard pe care o configurezi de fiecare dată (ex. foldere, controllere, modele)?
- Ce tip de operații CRUD implementezi frecvent în primele etape?
- Ce pași consideri că sunt cei mai repetitivi și consumatori de timp în etapa de setup?
- Există porțiuni de cod pe care le copiezi din proiecte anterioare pentru a accelera procesul?

- Te folosești de template-uri, scripturi sau unelte automate pentru a reduce munca repetitivă?
- Cum vezi integrarea unei aplicații de generare automată în fluxul tău de lucru? Ți s-ar părea utilă în mediul profesional sau academic?
- Care ar fi, din punctul tău de vedere, caracteristicile esențiale ale unei astfel de aplicații pentru a fi cu adevărat utilă?

Observația directă:

Am analizat propriul proces de lucru și am documentat pașii pe care îi parcurgeam în mod repetitiv la fiecare proiect nou, identificând astfel activitățile care pot fi automatizate fără a compromite calitatea sau flexibilitatea aplicației.

1. **Crearea soluției și a proiectului ASP.NET Core**
   - Deschiderea unui mediu de lucru (ex: terminal, IDE)
   - Generarea manuală a soluției și a proiectului (folosind dotnet new)
   - Alegerea template-ului potrivit (ex: webapi, mvc)
2. **Configurarea structurii de directoare**
   - Crearea folderelor pentru: Controllers, Models, Services, Data,etc
   - Organizarea logică a fișierelor pentru menținerea clarității în arhitectură
3. **Definirea entităților (models)**
   - Scrierea claselor pentru fiecare entitate (ex: User, Product etc.)
   - Definirea câmpurilor, tipurilor de date și relațiilor
4. **Crearea contextului bazei de date (DbContext)**
   - Scrierea clasei care moștenește DbContext
   - Înregistrarea entităților în context
   - Configurarea DbSet<> pentru fiecare model
5. **Configurarea conexiunii la baza de date**

- Adăugarea string-ului de conexiune în appsettings.json
- Configurarea serviciului în Program.cs

6. **Generarea și aplicarea migrărilor**
   - Rularea comenzilor dotnet ef migrations add și dotnet ef database update
7. **Crearea controlerelor pentru operațiunile CRUD**
   - Implementarea manuală a logicii de tip Create, Read, Update, Delete
   - Utilizarea Entity Framework pentru acces la date
8. **Adăugarea serviciilor în containerul de injecție**
   - Înregistrarea claselor în Program.cs pentru a fi injectate în controlere

Prin utilizarea acestor metode complementare, am obținut o imagine clară și bine fundamentată asupra cerințelor funcționale și nefuncționale ale sistemului, ceea ce a permis conturarea unei arhitecturi clare și dezvoltarea unor funcționalități relevante, conforme cu nevoile reale ale utilizatorului final.

2.4 Cazurile de utilizare

În această secțiune am realizat o diagramă de tip Use Case, cu scopul de a evidenția în mod clar interacțiunile esențiale dintre utilizator și sistemul informatic propus. Diagrama reprezintă o viziune de ansamblu asupra funcționalităților principale ale aplicației de generare automată a unui proiect ASP.NET Core.

Actorul principal este utilizatorul, care poate fi un programator sau orice persoană interesată să genereze rapid un proiect backend fără a parcurge manual toți pașii tehnici. Acesta interacționează direct cu sistemul, parcurgând următoarele etape, fiecare modelată ca un use case:

- Generare Proiect – funcționalitatea centrală a aplicației, care coordonează întreaga logică de generare a fișierelor. Acest caz de utilizare include:
  - introducerea numelui proiectului (*Project Name*)
  - introducerea numelui soluției (*Solution Name*)

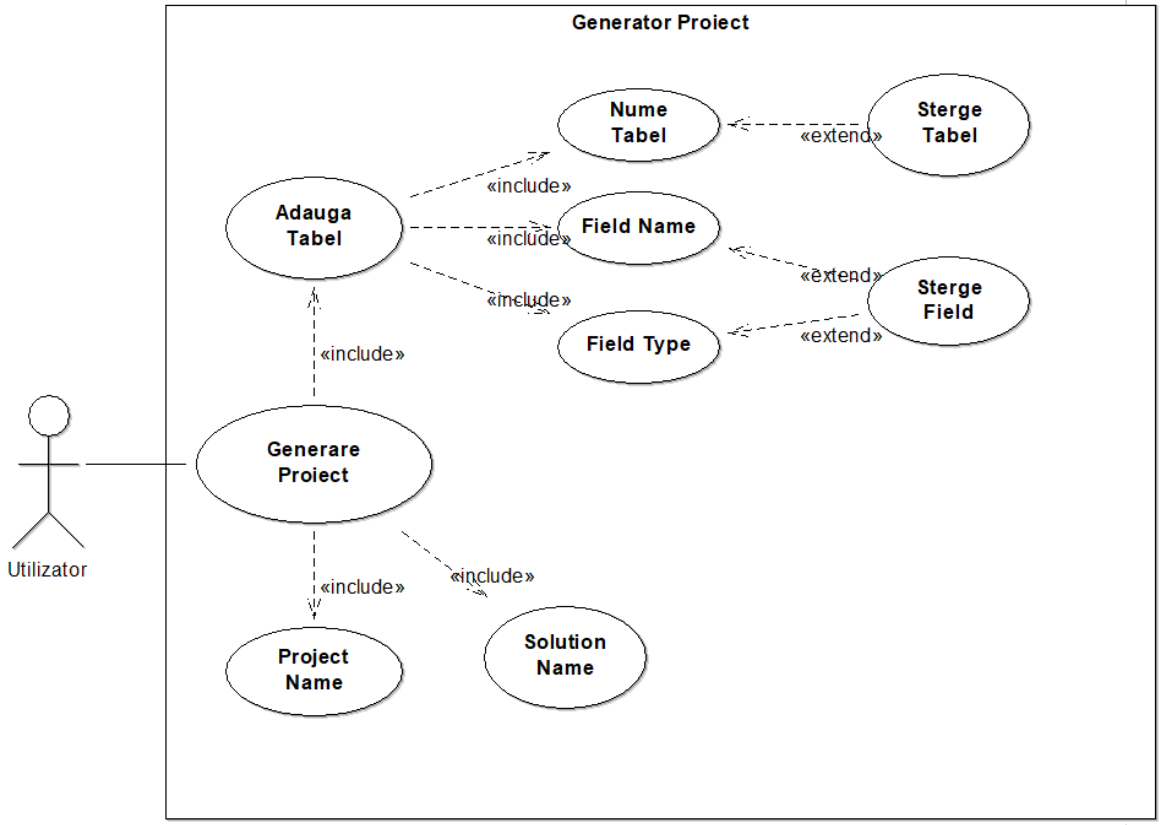
- Adăugare Tabel – etapă în care utilizatorul definește tabelele bazei de date necesare proiectului. Acest caz de utilizare include:
    - definirea numelui tabelului (*Nume Tabel*)
    - definirea câmpurilor (*Field Name*) și a tipului de date (*Field Type*)

Pentru o mai mare flexibilitate, sistemul permite și operațiuni suplimentare (extinse):

- Ștergere tabel – posibilitatea de a elimina un tabel anterior definit
- Ștergere câmp – eliminarea unui câmp adăugat greșit sau neutilizat

În dezvoltarea și utilizarea aplicației, am urmărit să acopăr cât mai fidel nevoile identificate în etapa de elicitație a cerințelor, rezultate în urma interviurilor realizate cu programatori, precum și a observației directe asupra propriului proces de lucru. Aceste metode mi-au permis să înțeleg clar care sunt pașii frecvent repetați, dificultățile întâlnite în faza de inițializare a proiectelor ASP.NET Core și funcționalitățile care ar aduce o valoare reală pentru utilizator.

Astfel, aplicația a fost concepută pentru a răspunde acestor nevoi concrete, printr-o interfață intuitivă, opțiuni de configurare flexibilă a tabelelor și câmpurilor, și generarea automată a codului necesar pentru funcționalități CRUD. Fiecare decizie legată de designul aplicației și structura sa funcțională a fost fundamentată pe cerințele extrase în această etapă, cu scopul de a reduce munca repetitivă și a eficientiza procesul de dezvoltare.



Figură 6 Diagrama cazurilor de utilizare

3. Proiectarea și Implementarea sistemului informatic

Așa cum am menționat anterior, scopul principal al aplicației dezvoltate este acela de a înlocui pașii manuali implicați în procesul de creare a unui proiect. Prin urmare, în acest capitol voi prezenta modul de implementare al aplicației, evidențiind metodele și tehnologiile utilizate pentru automatizarea etapelor repetitive din inițializarea unui proiect ASP.NET Core.

3.1 Structura generală a proiectului

Am hotarât să proiectez aplicația pe o arhitectură modernă bazată pe separarea clară între partea de backend și partea de frontend, așa că pentru backend am utilizat framework-ul ASP.NET Core care permite dezvoltarea de API-uri REST eficiente si oferă integrare nativă cu Entity Framework Core, un ORM performant pentru gestionarea bazelor de date relaționale. Partea de frontend am realizat-o folosind Angular, un framework JavaScript destinat contruirii de aplicații web de tip single-page (SPA).

Aplicația este formată din urmatoarele componente:

Backend

- Folderul Controllers:
    - conține fișierul ProjectGeneratorController.cs, acest fișier conține clasa ProjectGeneratorController care este responasbilă pentru gestionarea operațiunilor de generare automată a proiectului.

- Folderul Data
    - Conține fișierul AppDbContext

- Folderul Service care conține următoarele fișiere:
    - AppDbContextGenerator.cs
    - AppSettingGenerator.cs
    - ControllerGenerator.cs
    - Functions.cs
    - ModelsGenerator.cs

- ProgramGenerator.cs

Aceste fișiere sunt responsabile pentru generarea automată a noilor foldere și fișiere din proiectul creat.

- Fișierul appsettings.json
- Fișierul Program.cs

3.2 Folderul Controllers

După cum am menționat mai sus, folderul Controllers conține un fișier numit ProjectGeneratorController.cs, acest fișier este responsabil cu gestionarea operațiilor pentru generarea automată a fișierelor în noul proiect. În acest fișier am creat clasa ProjectGeneratorController unde am implementat 6 metode publice, acestea preiau datele furnizate de utilizator prin interfața aplicației și declanșeaza funcțiile din backend responsabile pentru generarea fișierelor. Metodele definite aici sunt toate de tip [HttpPost], întrucât operațiile implică modificarea sistemului de fișiere (scriere de fișiere sursă, creare directoare, actualizare de configurări etc.).

Metodele Controllereului:

1. **GenerateAppDbContext** – generează automat clasa AppDbContext, folosind structura tabelelor oferite de utilizator.

```
[HttpPost]
0 references
public async Task<IActionResult> GenerateAppDbContext([FromBody]  GenerateAppDbContextModel Model)
{
    try
    {
        DbContextGenerator.GenerateAppDbContext(Model.appDbSettingsPath, Model.numeleProiectului, Model.tabele);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Figură 7 Metoda pentru generare AppDbContext.cs

2. **GenerateProject** – creează întreaga structură a soluției (proiectul, folderele, pachetele, fișierele de bază).

```
[HttpPost]
0 references
public async Task<IActionResult> GenerateProject([FromBody]   GenerateProjectModel Model)
{
    try
    {
        GenerareProiect.CreateProjectFunctions(Model.solutionName, Model.projectName, Model.numeFolder, Model.tabele);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
```

Figură 8 Metoda pentru generarea Proiectului

1. **AppSettingUpdate** – modifică fișierul appsettings.json pentru a adăuga sau actualiza stringul de conexiune la baza de date.

```
[HttpPost]
0 references
public async Task<IActionResult> AppSettingUpdate([FromBody] AppSettingsModel Model)
{
    try
    {
        AppSettingGenerator.AppSettingUpdate(Model.projectPath, Model.numeleProiectului);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Figură 9 Metoda pentru generarea AppSettingUpdate

2. **WriteControllerContent** – generează automat controlerele CRUD pentru fiecare entitate introdusă.

```
[HttpPost]
0 references
public async Task<IActionResult> WriteControllerContent([FromBody]  WriteControllerModel Model)
{
    try
    {
        ControllerGenerator.WriteControllerContent(Model.controllersPath, Model.numeleProiectului,Model.tabele);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Figură 10 Metoda pentru generarea Controllers.cs

3. **GenerateModel** – creează fișierele .cs pentru fiecare model, în funcție de câmpurile specificate.

```
[HttpPost]
0 references
public async Task<IActionResult> GenerateModel([FromBody]   GenerateModelModel Model)
{
    try
    {
        ModelGenerator.GenerareModelContent(Model.path, Model.numeleProiectului, Model.tabele);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Figură 11 Metoda pentru generarea Models.cs

6. **GenerareProgram** – generează fișierul Program.cs, necesar pentru rularea proiectului.

```
[HttpPost]
0 references
public async Task<IActionResult> GenerareProgram([FromBody]   GenerareProgramModel Model)
{
    try
    {
        ProgramGenerator.GenerareProgram(Model.path, Model.numeleProiectului);
        return Ok();
    }
    catch (Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
```

Figură 12 Metoda pentru generarea Program.cs

Rolul controllerului este de a acționa ca un intermediar între model și vizualizare. Acesta gestionează datele introduse de utilizator, le procesează (prin apelarea funcțiilor corespunzătoare din Model) și determină ce vizualizare trebuie afișată. În esență, controlerul actualizează modelul pe baza datelor introduse de utilizator și spune vizualizării ce să afișeze. In acest proiect controllerul primeste datele trimise din frontend sub formă de obiecte json (numele proiectului, structura tabelelor, etc...), după care apelează metodele corespunzatoare din Folderul Service, care conține fișierele cu logica de generare automată a fișierelor (ModelsGenerator, ProgramGenerator, ...).

Ce mai este de reținut e faptul că în cadrul acestui fișier am definit și clasele care descriu datele de intrare primite de la utilizator(numele proiectului, numele solutiei, tabele, campuri, etc.), acestea functioneaza ca modele de transfer de date (DTO):

- GenerateAppDbContextModel – conține numele proiectului =, calea către fișierul de configurare și structura tabelelor.

- GenerateProjectModel – conține numele proiectului, numele soluției,numele folderului și tabelele.

- AppSettingsProgramModel, WriteControllerModel, GenerateModelModel, GenerareProgramModel , fiecare conțin datele necesare pentru a genera automat o anumită componentă a aplicației.

Un exemplu de cum arată o clasa:

```
1 reference
public class GenerateAppDbContextModel
{
    1 reference
    public string appDbSettingsPath { get; set; }
    1 reference
    public string numeleProiectului { get; set; }
    1 reference
    public Dictionary<string, Dictionary<string, string>> tabele { get; set; }
}
```

Figură 13 Clasa GenerateAppDbContextModel

3.3 Folderul Service

Folderul Service, este centrul acțiunii deoarece acesta conține toate fișierele unde se regăsesc generatoarele care contribuie la crearea automată a noului proiect.

Folderul Service dupa cum am menționat și mai sus conține următoarele fișiere:

- AppDbContextGenerator.cs
- AppSettingGenerator.cs
- ControllerGenerator.cs
- Functions.cs
- ModelsGenerator.cs
- ProgramGenerator.cs

Modul prin care este generat noul proiect:

Pentru a genera un proiect nou am hotărât să automatizez urmatorii pași:

- Deschiderea VS sau a consolei pentru a crea un nou proiect
- Crearea unui folder unde să fie plasat proiectul
- Crearea efectivă a proiectului de tip “webapi”
- Crearea unui folder cu numele Data
- Adăugarea unui fișier(AppDbContext) de tip file în folderul Data
- Popularea acestuia cu codul necesar și tabelele corespunzătoare cerințelor utilizatorului
- Crearea unui folder cu numele Models și popularea acestuia cu modelele corespunzătoare cerințelor utilizatorului
- Crearea unui folder cu numele Controllers și popularea acestuia cu datele aferente
- Adăugarea în Program.cs a codului necesar rulării
- Adăugarea în appsettings.json a ConnectionStringului necesar pentru conectarea la baza de date
- Adăugarea pachetelor necesare creeri unui nou proiect

Pentru a automatiza acești pași am hotărât să folosesc terminalul “Windows PowerShell” .

De ce PowerShell?

Deoarece este cel mai comun terminal la momentul actual pe toate dispozitivele cu un sistem de operare windows.

Câteva beneficii pentru utilizarea PowerShell sunt:

- Automatizarea completă a proceselor repetitive
    - PowerShell permite rularea de scripturi care pot crea, compila, porni sau publica aplicații .NET fără intervenție manuală. Astfel, pași precum generarea de migrări, update-ul bazei de date sau lansarea aplicației pot fi automatizați eficient.

- Integrare excelentă cu .NET Core și SDK-ul .NET
    - PowerShell oferă suport nativ pentru comenzi precum dotnet new, dotnet restore, dotnet build și dotnet ef, care sunt esențiale în ciclul de viață al unei aplicații .NET Core. Prin urmare, permite rularea acestor comenzi direct din scripturi, fără a deschide manual terminalul.

- Gestionarea fișierelor și directoarelor
    - PowerShell are o sintaxă puternică pentru manipularea sistemului de fișiere, lucru foarte util în proiecte .NET care presupun generarea automată de fișiere sursă, fișiere de configurare sau structuri de directoare (ex: proiectul tău de generare backend).

- Execuție multiplatformă
    - Cu PowerShell Core (versiunea open-source), scripturile pot fi rulate nu doar pe Windows, ci și pe Linux și macOS, ceea ce extinde portabilitatea aplicațiilor .NET care vizează platforme diferite.

- Automatizarea migrărilor Entity Framework
    - În contextul aplicațiilor .NET, PowerShell este frecvent utilizat pentru a genera sau aplica migrări cu Entity Framework Core (ex: dotnet ef migrations add, dotnet ef database update), simplificând actualizările bazei de date pe parcursul dezvoltării.

- Suport pentru DevOps și CI/CD
    - PowerShell este adesea folosit în pipeline-urile de livrare continuă (CI/CD) pentru proiecte .NET, unde poate automatiza build-urile, testarea, publicarea și livrarea aplicațiilor.

3.3.1 Fisierul Functions.cs

După cum am menționat mai sus, am folosit comenzi in terminalul Powershell pentru a creea un nou proiect. Pentru a automatiza acest pas si pentru ca să nu introduc manual fiecare comandă pe rand in PowerShell, am realizat un script care să ruleze automat fiecare comanda necesară creeari proiectului. Primul pas in realizarea acestei operatiuni a fost să ma documentez cu privire la procesul de creeare a unui proiect nou in terminal, astfel pentru a face această documentație am hotărât să creez manual un proiect prin introducerea codului necesar în terminal, am început prin a introduce codul pentru crearea proiectului, urmat de creearea fișierelor si adăugarea pachetelor. Am luat fiecare comandă folosită și am notat-o, ulterior le-am pus împreună și le-am adăugat în scriptul create în fișierul Functions.cs.

Comenzile folosite sunt urmatoarele :

```
cd Desktop

mkdir {numeFolder}

cd {numeFolder}

dotnet new sln -n {solutionName}

dotnet new webapi -n {projectName}

cd {projectName}

mkdir Data

New-Item Data\AppDbContext.cs -ItemType File

mkdir Models

mkdir Controllers
```

-Adăugarea Pachetelor-

```
dotnet add package Microsoft.EntityFrameworkCore

dotnet add package Microsoft.EntityFrameworkCore.Design

dotnet add package Microsoft.EntityFrameworkCore.Relational

dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

dotnet add package Microsoft.EntityFrameworkCore.Tools

dotnet add package Newtonsoft.Json

dotnet add package Swashbuckle.AspNetCore

Folosind aceste comenzi oricine poate creea un proiect în .Net, dar singura problemă este faptul că fișierele nu conțin codul necesar pentru creearea unui proiect cu funcționalități CRUD. Ci proiectul creat este unul foarte simplu fără funcționalitееăi extra. Ca sa ajungem la proiectul dorit am creeat generatoare care să completeze cu cod toate fișierele necesare.

```
public static class GenerareProiect
{
    1 reference
    public static void CreateProjectFunctions(string solutionName, string projectName, string numeFolder,
        Dictionary<string, Dictionary<string, string>> tabele)
    {
        string creareProiect = $@"
            cd ..
            cd ..
            cd ..
            cd ..
            cd ..
            cd Users
            cd Tarau
            cd Desktop
            mkdir {numeFolder}
            cd {numeFolder}
            dotnet new sln -n {solutionName}
            dotnet new webapi -n {projectName}

            cd {projectName}

            mkdir Data
            New-Item Data\AppDbContext.cs -ItemType File

            mkdir Models
            mkdir Controllers

            dotnet add package Microsoft.EntityFrameworkCore
            dotnet add package Microsoft.EntityFrameworkCore.Design
            dotnet add package Microsoft.EntityFrameworkCore.Relational
            dotnet add package Microsoft.EntityFrameworkCore.SqlServer
            dotnet add package Microsoft.EntityFrameworkCore.Tools
            dotnet add package Newtonsoft.Json
            dotnet add package Swashbuckle.AspNetCore
        ";

        StringBuilder sb = new StringBuilder(creareProiect);
```

Figură 14 Scriptul pentru generarea proiectului

3.3.2 Implementarea generatoarelor de cod

Astfel, dupa ce am creat proiectul folosind scriptul, următorul pas necesar pentru a realiza un proiect cu funcționalități CRUD a fost să adaug cod in conținutul fișierelor create și să editez codul necesar din Program.cs si appsettings.json. După o sesiune de gândire și căutare de idei, am ajuns la concluzia că cea mai potrivită variantă pentru a adauga cod și a edita conținutul fișierelor este utilizarea clasei StreamWriter.

Clasa StreamWriter:

Clasa StreamWriter din C# scrie caractere într-un flux într-o codificare specificată. Metoda StreamWriter.Write() este responsabilă pentru scrierea textului într-un flux. Clasa StreamWriter este moștenită de la clasa TextWriter, care oferă metode pentru a scrie un obiect într-un șir de caractere, a scrie șiruri de caractere într-un fișier sau pentru a serializa XML. (Chand, 2024)

StreamWriter este definit în spațiul de nume System.IO. StreamWriter oferă următoarele metode Write.

1. Scriere: Scrie date în flux.
2. WriteAsync: Scrie datele în flux în mod asincron.
3. WriteLine: Scrie un terminator de linie în șirul de text sau în flux.
4. WriteLineAsync: Scrie asincron un terminator de linie în șirul de text sau în flux

Metoda aleasa de mine pentru a scrie cod în fisiere este Metoda:

- StreamWriter.WriteLine()

Astfel clasa StreamWriter și metoda StreamWriter.WriteLine() le-am folosit în fișierele urmatoare:

- AppDbContextGenerator.cs
- AppSettingGenerator.cs
- ControllerGenerator.cs
- ModelsGenerator.cs
- ProgramGenerator.cs

3.4 Generarea Fișierelor

3.4.1 ModelsGenerator.cs:

În acest pas am utilizat clasa StreamWriter pentru a genera și popula fișierele Models bazate pe numarul de tabele precizate de utilizator. După ce am creat manual un proiect model care să utilizeze aceași structură pe care o realizez la proiectul automat, am preluat codul din fișierul unui model si prin metoda WriteLine am reprodus codul utilizând variabile și parametrii pentru a putea modifica modelele dupa preferinta utilizatorului.

```
public static class ModelGenerator
{
    1 reference
    public static void GenerareModelContent(string modelsPath, string numeleProiectului, Dictionary<string, Dictionary<string, string>> tabele)
    {
        foreach (var tabel in tabele)
        {
            using (StreamWriter writer = new StreamWriter(@$"{modelsPath}/{numeleProiectului}/Models/{tabel.Key}Model.cs"))
            {
                writer.WriteLine("using System;");
                writer.WriteLine("using System.ComponentModel.DataAnnotations;");
                writer.WriteLine();
                writer.WriteLine($"namespace {numeleProiectului}.Models");
                writer.WriteLine("{");
                writer.WriteLine($"    public class {tabel.Key}");
                writer.WriteLine("    {");
                writer.WriteLine("        [Key]");
                writer.WriteLine("        public Guid Id { get; set; }");
                foreach (var field in tabel.Value)
                {
                    writer.WriteLine($"        public {field.Value} {field.Key}" + " { get; set; }");
                }
                writer.WriteLine("    }");
                writer.WriteLine("}");
            }
        }
    }
}
```

Figură 15 ModelsGenerator.cs

Functia GenerareModelContent este apelată în controller în momentul în care utilizatorul generează un proiect nou (vezi Figura 8. Generare Models.cs), iar dupa ce metoda este apelată din controller se trece în folderul Service la fișierul ModelsGenerator.cs unde se crează modelele dorite de către utilizator .

3.4.2 AppSettingGenerator

În acest pas fișierul appsetting.json este citit și convertit într-un obiect JSON manipulabil, astfel se verifică existența secțiunii ConnectionStrings și dacă aceasta nu e prezentă, se adaugă. La acest element se inserează apoi un element DefaultConnection, care conține connection string-ul configurat. Iar la final obiectul este serializat din nou în format JSON și fișierul appsettings.json este rescris si conținutul vechi este înlocuit cu cel actualizat care include conexiunea la baza de date.

```
public static class AppSettingGenerator
{
    1 reference
    public static void AppSettingUpdate(string projectPath, string numeleProiectului)
    {
        string json = File.ReadAllText($@"{projectPath}/{numeleProiectului}/appsettings.json");
        var root = JsonNode.Parse(json)?.AsObject() ?? new JsonObject();

        if (!root.ContainsKey("ConnectionStrings"))
        {
            root["ConnectionStrings"] = new JsonObject();
        }

        root["ConnectionStrings"]![["DefaultConnection"] =
            $"Server =(localdb)\\mssqllocaldb;Database={numeleProiectului};Trusted_Connection=False;TrustServerCertificate=True;User ID=sa;Password=2410;";

        var options = new JsonSerializerOptions { WriteIndented = true };
        File.WriteAllText($@"{projectPath}/appsettings.json", root.ToJsonString(options));
    }
}
```

Figură 16 AppSettingsGenerator.cs

3.4.3 ControllerGenerator

În acest pas, se generează fișierele pentru controllere și se adaugă codul necesar fiecarui controller. Se folosește din nou clasa StreamWriter pentru a completa controlerele cu cod. Fiecare controller este construit în așa fel încat să includă funcționalitatea completă de tip CRUD(Create, Read, Update, Delete), prin definirea metodelor corespunzatoare pentru solicitarile HTTP (GET, POST, PUT, DELETE).

În figurile de mai jos se poate observa metoda WriteLine folosită pentru a scrie metodele de tip HTTP.

```
writer.WriteLine("        [HttpGet]");
writer.WriteLine("        public async Task<IActionResult> GetAll()");
writer.WriteLine("        {");
writer.WriteLine($"            var {tabel.Key} = await context.{tabel.Key}s.ToListAsync();");
writer.WriteLine($"            return Ok({tabel.Key});");
writer.WriteLine("        }");
writer.WriteLine();

writer.WriteLine("        [HttpGet]");
writer.WriteLine($"        public async Task<IActionResult> GetById(Guid id)");
writer.WriteLine("        {");
writer.WriteLine($"            var {tabel.Key} = await context.{tabel.Key}s.FirstOrDefaultAsync(x => x.Id == id);");
writer.WriteLine($"            if ({tabel.Key} == null)");
writer.WriteLine("            {");
writer.WriteLine("                return NotFound();");
writer.WriteLine("            }");
writer.WriteLine($"            return Ok({tabel.Key});");
writer.WriteLine("        }");
writer.WriteLine();
```

Figură 17 Solicitare HTTP de tip Get in controller

```
writer.WriteLine("        [HttpPost]");
writer.WriteLine($"public async Task<IActionResult> Create({tabel.Key} {tabel.Key}DTO)");
writer.WriteLine("        {");
writer.WriteLine($"            {tabel.Key} {tabel.Key} = new {tabel.Key}();");
foreach (var field in tabel.Value)
{
    writer.WriteLine($"            {tabel.Key}.{field.Key} = {tabel.Key}DTO.{field.Key};");
}
writer.WriteLine("        return Ok();");
writer.WriteLine("        }");
writer.WriteLine();
```

Figură 18 Solicitare HTTP de tip Post in controller

```
writer.WriteLine("        [HttpPut]");
writer.WriteLine($"        public async Task<IActionResult> Update(Guid id, {tabel.Key} {tabel.Key}DTO)");
writer.WriteLine("        {");
writer.WriteLine($"            var {tabel.Key} = await context.{tabel.Key}s.FirstOrDefaultAsync(x => x.Id == id);");
writer.WriteLine($"            if ({tabel.Key} == null)");
writer.WriteLine("            {");
writer.WriteLine("                return NotFound();");
writer.WriteLine("            }");
foreach (var field in tabel.Value)
{
    writer.WriteLine($"            {tabel.Key}.{field.Key} = {tabel.Key}DTO.{field.Key};");
}
writer.WriteLine($"            await context.SaveChangesAsync();");
writer.WriteLine($"            return Ok();");
writer.WriteLine("        }");
writer.WriteLine();
```

Figură 19 Solicitare HTTP de tip Put in controller

```
writer.WriteLine("        [HttpDelete]");
writer.WriteLine($"        public async Task<IActionResult> Delete(Guid id)");
writer.WriteLine("        {");
writer.WriteLine($"            var {tabel.Key} = await context.{tabel.Key}s.FirstOrDefaultAsync(x => x.Id == id);");
writer.WriteLine($"            if ({tabel.Key} == null)");
writer.WriteLine("            {");
writer.WriteLine("                return NotFound();");
writer.WriteLine("            }");
writer.WriteLine($"            context.{tabel.Key}s.Remove({tabel.Key});");
writer.WriteLine($"            await context.SaveChangesAsync();");
writer.WriteLine($"            return NoContent();");
writer.WriteLine("        }");
writer.WriteLine("    }");
writer.WriteLine("}");

writer.Close();
```

Figură 20 Solicitare HTTP de tip Delete in controller

3.4.4 DbContextGenerator

In acest pas, am utilizat clasa StreamWriter si metoda WriteLine pentru a genera conținutul fișierului AppDbContext.cs pentru a realiza conxiunea cu baza de date utilizand clasa AppDbContext. Scopul a fost configurarea automată a clasei AppDbContext, care gestionează conexiunea cu baza de date și definește accesul la entitățile aplicației prin intermediul DbSet.

```
public static class DbContextGenerator
{
    1 reference
    public static void GenerateAppDbContext(string projectPath, string numeProiect, Dictionary<string, Dictionary<string,string>> tabele)
    {
        using (StreamWriter writer = new StreamWriter(@$"{projectPath}/{numeProiect}/Data/AppDbContext.cs"))
        {
            writer.WriteLine($"using {numeProiect}.Models;");
            writer.WriteLine("using System.Collections.Generic;");
            writer.WriteLine("using Microsoft.EntityFrameworkCore;");
            writer.WriteLine("using Microsoft.Extensions.Configuration;");
            writer.WriteLine();
            writer.WriteLine($"namespace {numeProiect}.Data");
            writer.WriteLine("{");
            writer.WriteLine("    public class AppDbContext : DbContext");
            writer.WriteLine("    {");
            writer.WriteLine("        private readonly IConfiguration _configuration;");
            writer.WriteLine();
            writer.WriteLine("        public AppDbContext(IConfiguration configuration)");
            writer.WriteLine("        {");
            writer.WriteLine("            _configuration = configuration;");
            writer.WriteLine("        }");
            writer.WriteLine();


            foreach (var tabela in tabele)
            {
                writer.WriteLine($"        public DbSet<{tabela.Key}> {tabela.Key}s" + " { get; set; }");
            }

            writer.WriteLine();
            writer.WriteLine("        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)");
            writer.WriteLine("        {");
            writer.WriteLine("            if (!optionsBuilder.IsConfigured)");
            writer.WriteLine("            {");
            writer.WriteLine("                optionsBuilder.UseSqlServer(_configuration.GetConnectionString(\"DefaultConnection\"));");
            writer.WriteLine("            }");
            writer.WriteLine("        }");
            writer.WriteLine("    }");
            writer.WriteLine("}");
        }
```

Figură 21 DbContextGenerator

3.4.5 ProgramGenerator

În acest pas, am folosti aceași clasă StreamWriter pentru a edita fișierul Program.cs astfel încat toate funcționalitățile să fie implementate pentru funcționarea corectă a proiectului.

Am adăugat următoarele lini de cod in Program.cs:

- writer.WriteLine($"using {numeleProiectului}.Data;");
    - Importul namespace-ului pentru AppDbContext

- builder.Services.AddDbContext<AppDbContext>(ServiceLifetime.Transient);
    - Înregistrarea AppDbContext în containerul de servicii

- builder.Services.AddSwaggerGen();
- builder.Services.AddControllers();
    - Activarea serviciilor Swagger și Controller

- app.MapControllers();
    - Configurarea rutelor pentru controllere

- app.UseSwagger(c => {
  c.OpenApiVersion = Microsoft.OpenApi.OpenApiSpecVersion.OpenApi2_0;
  });
  app.UseSwaggerUI();
    - Configurarea interfeței Swagger UI

```
public static class ProgramGenerator
{
    1 reference
    public static void GenerareProgram(string path, string numeleProiectului)
    {

        string[] programLines = File.ReadAllLines($@"{path}/{numeleProiectului}/Program.cs");
        using (StreamWriter writer = new StreamWriter($@"{path}/{numeleProiectului}/Program.cs"))
        {
            writer.WriteLine($"using {numeleProiectului}.Data;");
            foreach (var line in programLines)
            {
                if (line.Trim() == "var app = builder.Build();")
                {
                    writer.WriteLine("builder.Services.AddDbContext<AppDbContext>(ServiceLifetime.Transient);");
                }
                if (line.Trim() == "builder.Services.AddOpenApi();")
                {
                    writer.WriteLine("builder.Services.AddSwaggerGen();");
                    writer.WriteLine("builder.Services.AddControllers();");
                }
                if (line.Trim() == "app.UseHttpsRedirection();")
                {
                    writer.WriteLine("app.MapControllers();");
                }
                if (line.Trim() == "app.MapOpenApi();")
                {
                    writer.WriteLine("app.UseSwagger(c =>");
                    writer.WriteLine("{");
                    writer.WriteLine("c.OpenApiVersion = Microsoft.OpenApi.OpenApiSpecVersion.OpenApi2_0;");
                    writer.WriteLine("});");
                    writer.WriteLine("app.UseSwaggerUI();");
                }

                writer.WriteLine(line);
            }
        }
```

Figură 22 ProgramGenerator

4. Testarea

În cadrul acestui capitol, mi-am propus să prezint modul și tehnicile prin care am efectuat testarea aplicației dezvoltate. Am încrecat testarea cât mai eficientă pentru a reduce semnificativ posibilitatea apariției unei erori, asigurând astfel o experiență de utilizare fără probleme.

Testarea software reprezintă verificarea în profunzime și sistematică a funcționalității unei aplicații sau a unui program. Funcționarea corectă a software-ului este determinată de testarea eficientă și corectă pentru a detecta tot felul de defecte care pot aparea pe parcursul dezvoltării unei aplicații sau a unui program. Sunt mai multe estape care alcatuiesc procesul de testare software, Ciclul de Viață al Testării Software (Software Testing Life Cycle STLC). Procesul de testare poate fii realizat la nesfârșit, verificând variabilele individuale automat sau manual.

Numarul de variabile este nelimitat și, prin urmare, nu există un produs testat 100%. Cu toate acestea, se poate încerca să se excludă cele mai probabile erori în unele condiții date.

În momentul în care testarea se îcepe chiar în prima fază a dezvoltarii unui program sau a unei aplicții, șansa ca produsul finit să fie de o calitate mult mai bună crește. Chiar din punct de vedere financiar, se economisesc banii care ar fii fost cheltuiți pentru posibile îmbunatațiri. Testarea crește rezistența si securitatea unui program, de exemplu poate exista un atac de hacking iar dacă aplicația sau programul nu a avut sistemul de securitate bine testat ar putea duce la o pierdere enormă.

Este esențial să înțelegem că testarea software se desfășoară pe mai multe niveluri, fiecare corespunzând unei etape diferite din procesul de dezvoltare. Cele mai comune niveluri de testare includ: testarea unitară, testarea de integrare, testarea la nivel de sistem și testarea de acceptare.

Tipurile de testare pot fi efectuate la fiecare nivel de testare software. Câteva tipuri de testare sunt: teste structurale, teste legate de modificări și teste funcționale și non-funcționale.

Testele funcționale, verifică funcționalitatea unei aplicații. Sunt mai simplu de realizat decât testele non-funcționale. Testele non-funcționale se concentrează pe modul în care funcționează un o aplicatie sau un program, dacă este eficace, eficient, sigur și ușor de înteles.

Testele structurale testează codul și încearcă să găsească erori pe baza acestuia. Se efectuează teste de regresie (un tip foarte important de teste), de exemplu, după efectuarea unei reparații în codul unui program. Testul este apoi efectuat ca să determine dacă programul a reacționat la schimbare în maniera dorită.

Luând în considerare tipurile de teste, trebuie să menționăm diferența dintre testarea software și testele de asigurare a calității (Quality Assurance – QA). Testarea software permite detectarea erorilor într-un anumit software, iar performanța lor nu necesită o bună cunoaștere a codului. Testele QA, servesc mai degrabă la optimizarea calității unui anumit produs, indiferent de eventualele defecte. Aici, cunoașterea si înțelegerea codului este necesară, deoarece testerul de calitate poate sugera posibile îmbunătățiri la codul aplicației

Testarea software este o activitate aparent de rutină și plictisitoare dar foarte necesară pentru orice tip de aplicție sau program. O aplicație care nu a trecut printr-un proces de testare este un risc pentru viitorul clien. Testării joaca un rol crucial in fiecare companie dezvoltatoare de aplicații, totuși unele firme decid sa nu angajeze testeri ci sa își utilizeze developerii pentru a realiza testele, aceasta practică e una foarte riscantă. (academy, 2025)

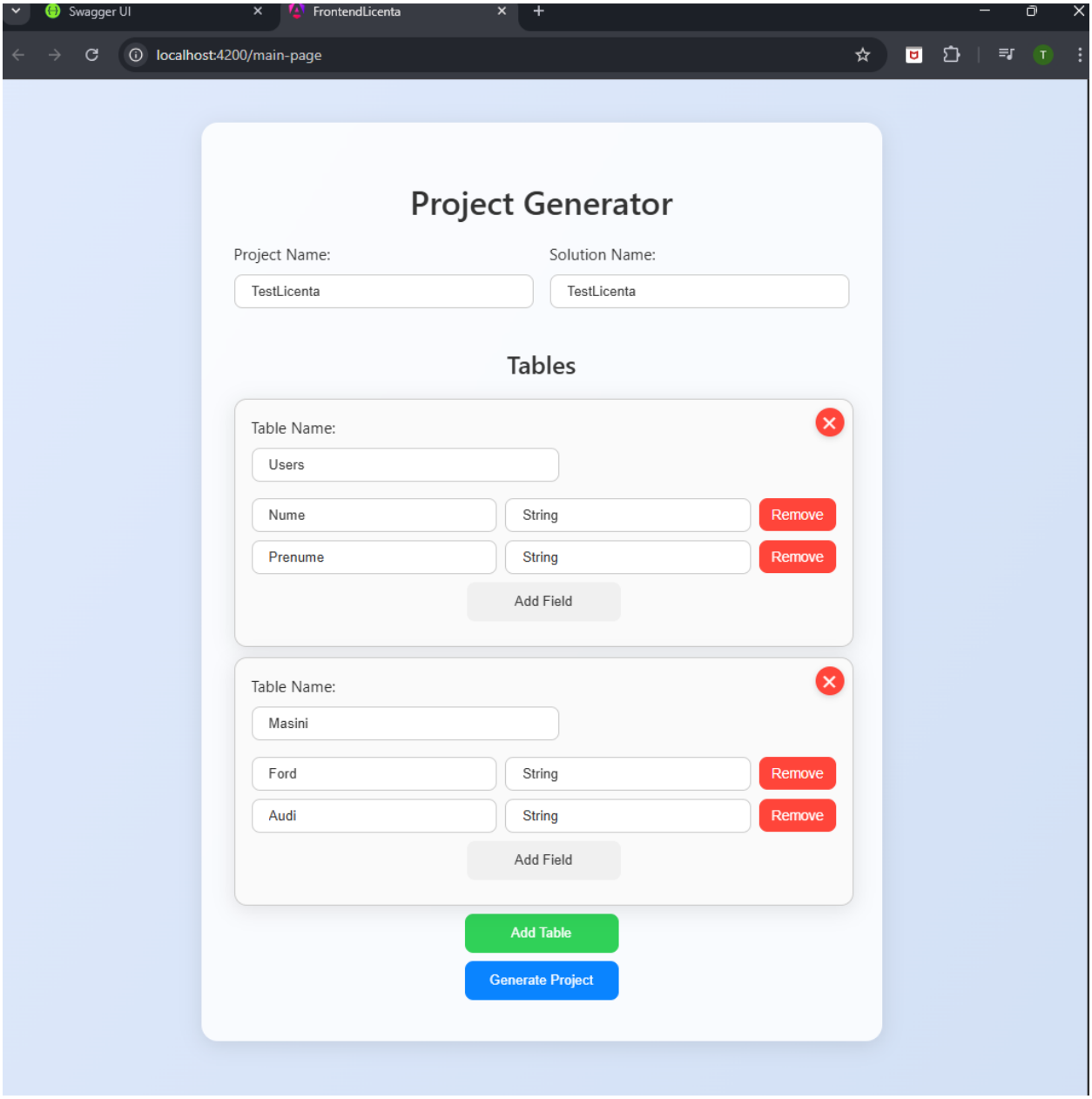
Avand in vedere metodologia folosita pentru a crea aplicatia (Waterfall), etapele de testare au fost facute treptat.

Pentru teste pe partea de server au fost efectuate urmatoarele operatiuni:

- Testare manuala folosind tool-uri de testare și apelare de API-uri precum Swagger

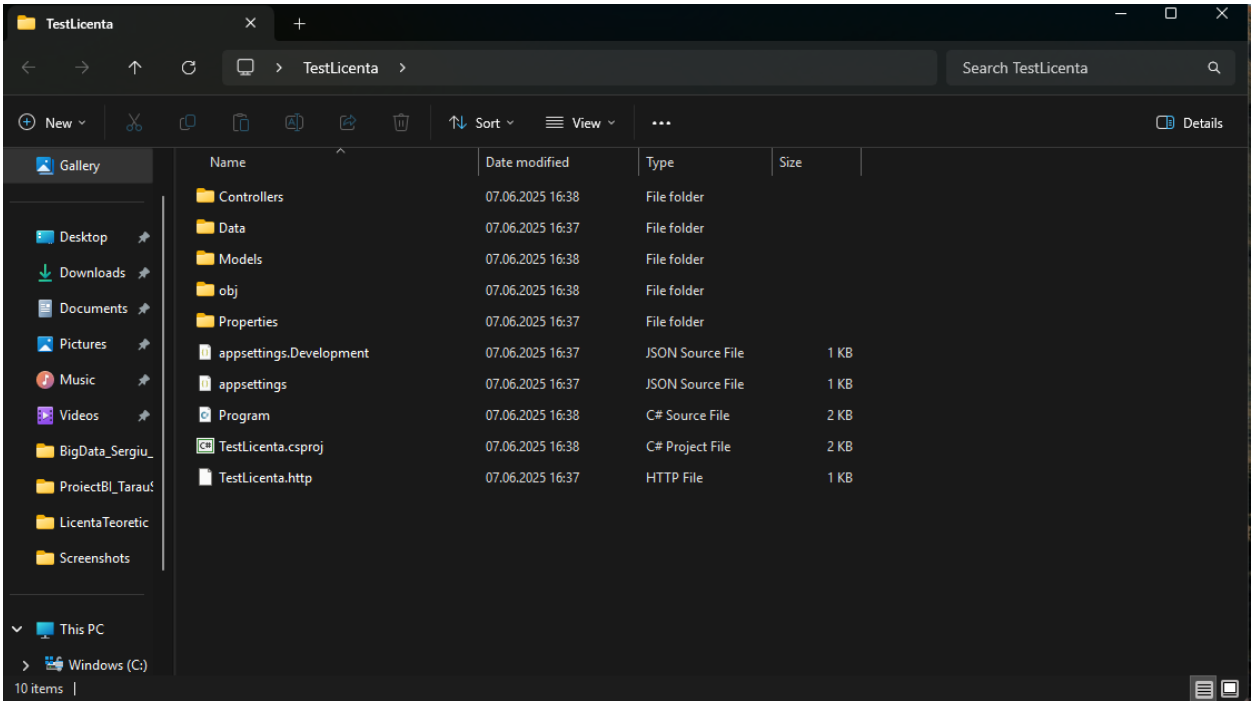
Scenariu de testare

ID	Test Case 1
Denumirea	Generarea proiectului
Prioritate	Ridicată
Rezultatul așteptat	Generarea corectă a proiectului
Scop	Verificarea funcționalității de generare a proiectului automat
Autor	Tarau Sergiu
Pași	1.Deschiderea aplicației web 2.Introducerea datelor necesare creeri unui proiect 3.Apasarea butonului Generate Project
Rezultat	1. Proiectul este generat



Figură 23 Test Case 1

Rezultatul Testului:



Figură 24 Rezultat Test Case 1

Putem observa faptul că aplicația funcționează corect și proiectul a fost creat, cu toate fișierele și funcționalitățile.

5. Concluzii

5.1 Rezultate obtinute

Având în vedere contextul de pornire și obiectivele, putem afirma cu încredere că dezvoltarea unei asemenea aplicații de automatizare a proceselor manuale a avut o utilitate indiscutabila și a adus beneficii multiple pentru fiecare pasionat de programare sau programator care își dorește să eficientizeze timpul alocat unui proiect și să reducă costurile. Astfel, am reușit să aducem o rezolvare problematicilor identificate pe parcursul lucrării.

În urma unei analize atente a nevoilor utilizatorilor, am observat și identificat pașii care se repetă constant în cadrul fiecărui proiect de tip .NET și am propus o soluție care să înlocuiască acești pași manuali. Astfel aplicația creată permite utilizatorului să defineasca structura de tabele și câmpuri printr-o interfața simplă, iar în urma acestor date, sistemul generează automat codul sursa necesar pentru un proiect de tip ASP.NET Core cu funcționalități CRUD.

În realizarea acestui proiect am folosit clase precum StreamWriter pentru a scrie automat cod în fișierele .cs, iar PowerShell l-am folosit pentru a executa comenznile sistematice, cum ar fii crearea de directoare sau inițializarea unei soluții.

În ceea ce privește rezultatele cercetarii, pot să confirm faptul că utilizarea tehnologiilor .NET, C#, Entity Framework, JSON, Swagger, Dependency Injection, Angular în dezvoltarea aplicațiilor web pentru automatizare a proceselor manuale este eficientă si potrivită.

5.2 Directii viitoare

După cum am menționat și în capitolele anterioare, nevoia de astfel de aplicații este tot mai mare, atât din punct de vedere al eficientizarii timpului dar și prin faptul că piața duce lipsă de astfel de aplicații, numarul lor fiind unul restrâns. Așadar, direcțiile de dezvoltare ale acestei aplicații sunt nelimitate, acest timp de aplicație poate fii dezvoltată pe foarte multe ramuri, cum ar fii:

- Implementarea dezvoltarii și altor proiecte (nu doar ASP.NET Core)
- Implementarea automatizarii migrațiilor
- Implementarea automatizării proceselor viitoare din aceasta aplicație
- Adăugarea de noi funcționalități pentru tabele
- Implementarea unui sistem care să dezvolte automat și frontendul
- Implementarea unui AI care să ajute la dezvoltarea aplicației

(OpenAI, 2022)

Bibliography

(fără an). Preluat de pe https://flexi-project.com/ro/ce-este-metodologia-waterfall-si-cum-functioneaza-aceasta/

academy, s. d. (2025, 02 11). *Ce altceva ar trebui să știți despre testarea software?* Preluat de pe https://sdacademy.ro/blog/ce-altceva-ar-trebui-sa-stiti-despre-testarea-software/

Chand, M. (2024). *C# StreamWriter Example*. Preluat de pe https://www.c-sharpcorner.com/article/csharp-streamwriter-example/

Development, R. (fără an). CE ESTE ANGULAR? Preluat de pe https://www.roweb.ro/ro/tehnologii/angular

DevelopmentFactory. (2024, 09 21). *MVC*. Preluat de pe https://developmentfactory.ro/blog/mvc-design-pattern

Dominik, W. (fără an). Ce este metodologia Waterfall? Preluat de pe https://flexi-project.com/ro/ce-este-metodologia-waterfall-si-cum-functioneaza-aceasta/

GeeksForgeeks. (2025, 01 03). *Modelul de design MVC* . Preluat de pe https://www.geeksforgeeks.org/system-design/mvc-design-pattern/

GeeksForgeeks.org. (2025, 01 03). *MVC .* Preluat de pe https://chatgpt.com/c/68538a85-d11c-800c-9185-0b22b32ac045

Ilie, S. (2024 , 08 19). Ce este un API? Forma completă, semnificație, definiție, tipuri și exemplu. Preluat de pe https://www.guru99.com/ro/what-is-api.html

Microsoft. (2022, 07 22). Entity Framework. Preluat de pe https://learn.microsoft.com/en-us/aspnet/entity-framework

Microsoft. (2024, 11 24). ASP.NET Core. Preluat de pe https://en.wikipedia.org/wiki/ASP.NET_Core

Microsoft. (2024, 10 31). *What is PowerShell?* Preluat de pe https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.5

Microsoft. (fără an). Entity Framework (EF). Preluat de pe https://en.wikipedia.org/wiki/Entity_Framework

Mozzila. (fără an). *Working with JSON*. Preluat de pe https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON

npmjs.com. (2025). *TypeScript*. Preluat de pe https://www.npmjs.com/package/typescript

OpenAI. (2022, 11 30). *Introducing ChatGPT*. Preluat de pe https://openai.com/index/chatgpt/

Glosar

Waterfall – Model secvențial de dezvoltare software în care etapele (analiză, proiectare, implementare, testare) sunt parcurse în ordine fixă.

Framework – Ansamblu de biblioteci și reguli care oferă o structură standard pentru dezvoltarea aplicațiilor software.

Aplicație Web – Program care rulează pe un server și este accesat prin intermediul unui browser web.

Cross-platform – Capacitatea unui software de a rula pe mai multe sisteme de operare fără modificări majore.

Entity Framework – Framework ORM pentru .NET care permite interacțiunea cu baze de date relaționale folosind obiecte din cod.

Object-Relational Mapper (ORM) – Tehnologie care mapează obiectele aplicației la tabele dintr-o bază de date relațională.

Software – Set de programe și instrucțiuni care controlează funcționarea unui calculator sau dispozitiv.

Backend – Partea invizibilă a unei aplicații care se ocupă de logica, procesarea datelor și interacțiunea cu baza de date.

Frontend – Partea vizibilă a unei aplicații cu care interacționează utilizatorul (interfața grafică).

Browser – Aplicație software care permite navigarea pe internet și afișarea conținutului web (ex: Chrome, Firefox).

Dependency injection – Tehnică de programare prin care dependențele unui obiect sunt furnizate din exterior, pentru a spori modularitatea și testabilitatea.

Data-binding – Procesul de conectare a interfeței grafice cu datele din aplicație, permițând sincronizarea automată a acestora.

Bază de date – Structură organizată pentru stocarea și gestionarea datelor, accesibilă prin interogări.

Shell – Interfață de linie de comandă care permite utilizatorilor să interacționeze cu sistemul de operare.

Terminal – Fereastră de linie de comandă unde se pot executa comenzi direct în sistemul de operare.

Sistem informatic – Ansamblu de echipamente, software și persoane care lucrează împreună pentru a colecta, procesa și stoca informații.

Migrații – Mecanisme de versionare a modificărilor aduse bazei de date, utilizate în timpul dezvoltării aplicațiilor.

Folder – Structură de organizare care poate conține fișiere sau alte foldere.

Fișier – Unitate de stocare ce conține informații digitale, cum ar fi cod sursă, date sau configurații.

Request – Cerere trimisă de un client către un server pentru a obține sau trimite informații.

Windows – Sistem de operare dezvoltat de Microsoft, utilizat pe scară largă în calculatoare personale și enterprise.

Script – Fișier care conține un set de instrucțiuni ce pot fi executate automat de un interpretor.

Clasă – Șablon în programare orientată pe obiecte care definește proprietățile și comportamentele unui obiect.

Metodă – Funcție definită într-o clasă care descrie o acțiune sau comportament al obiectului.

ConnectionString – Șir de caractere folosit pentru a stabili conexiunea între aplicație și baza de date.

Scenariu de testare – Descriere a unei situații specifice pentru a verifica funcționalitatea unui sistem.

Funcționalitate – Capacitatea unui sistem sau aplicații de a îndeplini o acțiune sau sarcină specifică.